

Space Invaders GL: Dev Logs

Voyager

2026/05/6

Introduction

This documents aims to record the summary of every session developing this program. The program will be a 3D gas particle diffusion simulation. The program will allow the changing of certain parameters in runtime to allow different outcomes. For example the sum energy of the system, particle size, particle sum and maybe energy loss multipliers could be added aswell. I aim to finish this project before the second term of AS.

I am giving myself quadtrouple the time I predict I need based on how fast "SpaceInvadersGL" was finished. That is because currently I am also learning "ESP-IDF" programming and reading subjects student books, so the time and mental capacity I can spend on the project is lesser. I will also make this project more polished, as in the code quality and interfaces.

My main goal of this project is to learn more about the relationship between math and physics, helping me not just use, but understand the math and physics concepts I have learnt.

2026/05/11

7:2:41

I have spent the last 5 days mostly developing for the esp32 than this project. I have finished the shader class, window creation and linked the necessary files. However, I have not displayed a triangle yet. I will finish it today so I can start developing the physics side of the program.

2026/5/12

13:51:55

I got tired of fixing dumb stuff (uniform assignment before shader bind, vao-vbo mix, shader false debug) instead of actually starting the project. So I put it all in claude so it could point me to every bug, so I did not have to spend multiple hours looking through the code to see the difference between an 'b' and a 'n',

I believe the projects code is stable enough to proceed. First I will have to figure out how I want to render the program. I know 3D physics will be much harder but I will go through with it. My biggest concern is that without shadows, the spheres depth would be hard to see. I will have to make every particle a different color so when particles overlap during rendering, it will not look like they merging/splitting.

I want to make the particles spheres, so the particles hitbox can be implemented simply using its radius, other particles radii and the distance from other particles origin. I believe this is where I have to optimize the most to support more particles without melting my laptop thing.

I will have to make a barrier so the particles don't diffuse off screen. I have not thought much about implementing the boundary but I believe there are 2 suitable methods. The first is the typical one, where I have to use a system of linear inequalities to describe an area in world space. If a vertex crosses this described area, then we know a collision has occurred with the boundary. This is just theoretical. I have never implemented use of systems of linear inequalities before but I don't see why it will not work

The second method makes a spherical boundary. The length between particles world space origin and the world space's origin is calculated. If the length is or is greater than a defined parameter, then we know a particle has gone off bounds and thus collided with the wall.

I will implement the systems of linear inequalities boundary since a spherical boundary is not standard and there may be complications with continuous collision if the particles came at the right angle.

To render the spherical particles is another challenge. There is probably a million tutorials out there but I will first try with my own mathematical capacity. I have 2 ideas in mind. Both ideas revolve around the use of making rings by recursively linearly interpolating with a linearly changing convexity factor. I don't know if that was the right word to describe this method. I am basically just repeatedly creating a vertex between 2 vertices and displace it outwards a certain amount, the displacement magnitude changes every instance. Without the convexing, the line would stay straight. Without changing the displacement factor, every interpolated vertex would displace out the same, creating a sort of fractal

Could I just hardcode a spheres vertex data to avoid all this? Yes, but then the spheres vertex count wouldn't be easily variable which is crucial for optimization. And it would be very tedious.

With this method, I can make a 2d ring. To generate the sphere, I can either move the "vertex generation ring" up and down while linearly changing the... It cannot be linear. Okay, my bad. I cannot generate circle with linear change, they have to be non-linear. So the convex factor of ring generation also cannot be linear. Okay, I can just change the "linear" parts of things and get the desired outcome either way. No need to rethink it. I can non-linearly change the diameter of the vertex generation ring to converge to a point up and below, making the sphere.

The second method to manipulate the vertex generation ring is to linearly

interpolation rotation axis instead of spatial axis. I believe this method is better since it is much simpler. It will be less optimized since overlapping vertices would be duplicated. Maybe I can figure out how to delete these "would overlap" vertices before generation.

Overall the graphical implementation seems tough. Especially since I need to work with non-linear trends which I am not very familiar with. I will still attempt to do it. If I fail, I will see some guides.

2026/5/13

17:34:19

I did not do any programming today. I mostly spent the day finishing learning C for esp-idf. I spent some time trying to figure out where to start and thought I would try to figure out what to input to generate the sphere. I settled on the origin planar vertex rings vertex count. I tried to get an equation to find the resultant sum of all vertices on the sphere after the generations and removal of coinciding vertices. I was close but then realized there is a more efficient way to generate.

The original method involved first generating a vertex ring and then "rotating" the generation ring so it is orthogonal to the first ring. The generation ring would generate a ring per vertex on the first ring. This has some inefficiencies.

First the orthogonal generation ring only has to generate a ring for half the vertices for even original vertices count and the greater half for an odd vertex count on the original planar ring. Secondly, at some point, every vertex on the original ring would be intercepted by the orthogonal rings generation, so we can skip the first part of the method. We can simply rotate the generation ring around one rotation axis which is not the axis which the original ring lays on. The only intersections then would be the upper and lower most vertices.

2026/5/14

19:58:43

I started work on the vertex ring generator. Right now the function can populate an empty array at the correct positions. I made sure to use variables as much as possible so I would not have to manually change constants much.

An issue now is the calculated values for the points. The first point at (1, 0) is calculated and put in the array correctly. However the vertices after that are not right. I got seemingly irrational decimal values. They were between 1 and -1. The angle value is definitely correct, I output it multiple times to check.

I then remembered the cmath libraries functions use radians as its standard rotation unit. I used the glm radians function to convert the angle degrees, but I still got incorrect values. Now the values are beyond the 1 to -1 range.

20:44:19

I got as far as I could into the problem on my own. I pin pointed the problem to the trig functions. I suspected it was some floating point precision thing so I googled it, found nothing. So I "Clauded" it and I confirmed it was infact the floating points precision causing it.

Pi being an irrational value cannot be stored in a finite memory space. Even if I made the "conversion space" a double datatype, It should not work. The effect of this on the graphical output would be negligible. Especially if the calculations are done with double data type. But it still bugs me so so I will implement a range within which any value would become a 1 or 0.

2026/5/15

6:26:50

Added a range in which a vertex coordinate becomes a 1 or 0.

17:27:40

Okay now the floating point precision error correction section works. To keep the code clean, I thought I would use the ternary operator to select the right operation. The function does look smaller but I am not sure if it is too "complex looking". I checked the absolute values and used the "copysign" function to give the proper sign for the 1's.

I also finished a temporary ebo generator function to see how well my functions hold up against different test data. I want to make my vertex ring generator alorythm to be robust before I move forware to 3D. Tommorow, I will have to integrate ebo usage into the program and test the functions.

2026/5/16

16:38:19

The vertex ring generator function is fully operational. I tested it with multiples of 4, a even non-multiple of 4 and a odd number, It passed all tests.

When I was wrote code before, I tried to abstract as much as possible, but without going out of the way to do it. But I noticed the some code looked worse due to some abstractions. I am refering to making the vertex buffer objects capacity a constant in the main function and passing it to the needed functions. The thing was the calculation was very simple. It was just the amount of vertices into the amount of vertex attribute data per vertex. I felt that reducing the word count of the function signature made the code more readable than repeating a small line of code across 2 functions.

This change fit in nicely as I was also passing the number of vertex attributes per vertex into necassary functions since the current value (6), was getting

repeated alot. I did not think this was a issue before since the number of vertex attribute data seemed fine then (3 coordinates, 3 color values/RGB). After some thought, I worry that the particles depth may not be emphasized enough with just solid colour and projection. I may have to learn basic lighting just for this project. I am not sure if that adds more depth to the vertex array object. That is why I made vertex attribute count a constant variable in the main function. Even without lighting I can think of adding more depth by changing the opacity of particles based on how overlapped it is. I would think that implementation would require something like alpha data for vertices.

Next I will have step into the third dimension.

2026/5/17

12:55:42

I will move to 3D in 2 steps. First I will setup the camera to be fully operational. The camera will be fixed on a single point in space, probably the center of the simulation "simulation box". It will be able to rotate to variable angles and zoom in and out. All done preferrably with just the mouse.

Next I will start work on rendering the sphere. I got most of the section abstracted but I am not too sure how to generate an ebo. Maybe I can figure out a pattern like before.

Splitting the transition to 2 stages will decrease the overall development time. Since any error I get I know is related to the current stage. If I worked on both the camera and sphere at once, there are alot more possible errors to check since any one of the 2, or even both may have caused the error.

16:51:17

I am done with setting up the camera. It is controlled via the keyboard. Spending extra time on a camera that would barely be moved is a waste. I know enough about projection matrices to implement them without issue. At first I was going to use a cube to test the camera system. It seemed simple enough but when I got to it, I realized had to manually enter alot of element buffer object data. So I opted to procedurally generate the cube like I did the sphere but again, generating the the element buffer object was the hard part. I could not figure out a suitable pattern with 4 vertices so I then planned to make all the vertices overlap with another. So there was a 4 per face. At that point I realied it was a bit much for some simple testing and scrapped the idea and used the already working vertex ring to test it.

I spent alot of time on the cube but I don't yet regret the time spent since it gave me further insight into element buffer object generation. Somethign I will have to face at some point to connect the full sphere.

Next I will start work on generating the vertex data for the full sphere. I have explained my plans for the algorithm before but now I might have to try

another way to accomodate the element buffer object generation. The original algorithm is simpler (just rotate the vertex ring generator)but leaves the vertices generated in a more fragmented order.

The only alternate method I have come up with so far is alot more complicated. It involves translating the vertex ring generator so the sphere consists of "stacks" of vertex rings. There are 2 variables that contribute to moulding this stack into a sphere. The distance between layers and radius of vertex rings. The variables can be adjusted in 2 ways from what I can think of.

The distance between vertex ring layers can remain constant while the radius of vertex rings non-linearly changes. Or the distance between vertex ring layers can non-linearly change while the radius of vertex rings linearly changes. I prefer the former method since one variable stays constant

Either way, there is a non-linear factor to deal with. I will think a bit more before I proceed.

2026/5/18

16:32:12

I have thought of another way to generate the sphere. No non-linear things involved. The algorithm will go like so.

First I will generate vertex rings, from top to bottom creating a stack. This ensures the ebo can be easily generated since the first vertex is at the very top. I have yet to make a equation to relate the vertices count of a single ring to the number of rings and Z distance between each ring. Maybe I can keep them seperate parameters?

Next the vertex rings must be shrunk relative to its distance from the center, getting smaller at the top and bottom. A single scaler can be calculated for every vertex ring, which when applied to every vertex of the appropriate ring, should have shrunk the vertex rings diameter.

Only one vertex's coordinate has to be used to calculate the scaler required for the rest of the ring to shrink. The Z coordinate of the vertex is used to calculate the x,y coordinates at the same Z coordinate in a sphere with a defined radius.

The only problem with this algorithm is that the top and bottom connecting vertices must be seperatly created. Doing so could make generating the ebo difficult but I see no other option as good as this one. To get started I will need to first derive the appropriate mathematical formulas to procedurally get some variables (distance between rings, number of rings)

17:40:16

I have encountered my first roadblock. I was trying to determine how to get the number of layers relative to the number of vertices per ring. I first tried manually creating vertices in a sphere arrangement in a online 3d modelling tool.

It got tedious very fast. Then I realized the number of layers was the number of unique y or x coordinate on the ring. I started to manually count the for ring with multiples of four but it got tedious, so I added a bit of code in the vertex ring generation function to increment a variable that is later output everytime a negative x coordinate was entered into the buffer. This it was even at the time, I did not have to care about the other side. Even when I tried odd numbers the pattern seemed to be straightforward.

$$Even|Layers = \frac{VerticesCount}{2} - 1 \quad (1)$$

$$Odd|Layers = \frac{VerticesCount}{2} + 1 \quad (2)$$

But then I actually looked at the output and saw issues. When the number of vertices in a ring were odd, there was no symmetry. There would always be a side with 1 more vertex. This was a pretty obvious result, I am not sure why I did not think of it. I will try to simulate manually simulate odd values since the previous algroythm should still work.

I was doing further data gathering when I entered in 15 vertices, returning a error. "malloc(): invalid size (unsorted)" followed by a core dump. I haven't encountered this specific error messege ever. I will try to look into it my self before searching it online.

2026/5/16

15:05:31

I have tried values from 1 to 32 to find problematic situations. The values 3, 15, 19, 23, 27, 31 return some kind of error. 3, 15 and 19 immediatly return a malloc(): invalid size error before aborting. The others return a size(): invalid size error, Both mean the same thing which I believe is writing outside allocated memory. Something different about 23 and 27 is both only return an error when I close them. I am not sure what the problem could be since there is seemingly no suitable pattern to fit the numbers. They are all odd and except 15, prime. But there are other odd primes that work liek 7 and 11.

I also thought about how odd numbers would generate layers and remembered that in the original algorythm involving spinning the vertix ring generator, starting with odd numbers would create a extra vertex making it even. But the distance between the new vertex would be inconsistent with the rings original vertices. If I cannot figure out how to make every number work, then I will validate the inputs to be only even numbers.

16:04:37

I will discontinue trying to make every number work. I will continue with even numbers.

With even numbers I see 2 results. One where the top and bottom vertices are alone and ones where there are 2 vertices on the same level at the top and bottom. I will accomodate for both.

2026/5/20

15:02:59

I was checking patterns to figure out how to differentiate between "lone" top and bottom vertices and paired when I noticed the distance between vertices relative to a straight line was not the same. I double checked by drawing on a screenshot of a vertex ring and confirmed that I could not keep the Z distance constant. The trend seemed to be that near the to and bottom, the Z distance between vertices would rapidly decrease. The large part of the middle vertices Z distance stayed relatively constant. A non-linear trend. I hope in AS or AL I will get more familiar with non-linear trends since this is getting annoying.

Something else I realized was that I had procedurally find out the number of layers. I could not just pass in the number of layers since then the smoothness would vary, making it assymetrical from most angles

Finally I figured something else out. It is like a hybrid of my first and last concept. I will generate vertices using 2 angles. The first angle (Yaw) determines the position of vetex relative to other vertices in its ring. The other angle (Pitch) determines the Z value of the vertex. This value will be constant for every vertices on a ring. The angle delta should remain same for both angles. The hard part is calculating this angle delta.

2026/5/21

13:43:29

I doubt I will be able to program anything today since I plan to finally learn github for this. I mainly want version control but I have also needed to learn github to start contributing to open source projects.

I am also done planning the math side of the implementation and have a formula to find the number of vertices required.

$$TotalVertexCount = VerticesInRing \cdot LayerCount + 2$$

The +2 accounts for the first and last vertices of the sphere. Originally I thought only spheres with "lone" vertices at the end layers would need to account for this. However, I realized even for "paired" layers, They need a middle point to converge to in the element buffer object.

2026/5/22

15:07:44

I still have not done major progress on the sphere generation, just more math. But now I am sure I have everything I need on the math side on things. Like formulas and functions.

I did write a bit of code and noticed the function signatures were getting pretty long. Furthermore, there are a lot of constants that are declared at the start of the functions. This led to me thinking of just creating a vertex buffer object class. It would have all the necessary attributes like vertex count, vertex attribute count per vertex, the vertex buffer object itself and more. This would allow me to just pass an instance of a vertex buffer object into all the functions. It would also look much neater since it would be obvious what the data structure is describing.

This would also allow the existence of different types of particles. hmmm, now that I think about it, implementing a vertex buffer object class is actually a really good idea. My only counter argument was it made calculations using the vertex buffer object's attributes less neat since I had to reference both object and attribute instead of just the attribute. That would also make the code more readable?

I will try to finish implementing the whole thing in a class this evening.

2026/5/23

17:36:36

Note: Here I talk about the number of vertex attributes per vertex, stride and vertex attribute count. I mean the same value (currently 6) in writing.

I thought I would be able to finish implementing the variable layer z value based on pitch but I had run into an issue. I implemented the loop and by dumping the vertex buffer object's data I can see the loop does work as expected with the vertices z value changing every layer at expected intervals and magnitudes.

However, only one vertex ring is output. I am not sure which layer it is but it does not seem to be the middle layer since the constructed rotation point is not at or near its center.

I was modifying the function that outputs the vertices data and noticed one set of vertex attribute data was missing. I had to increase the range of the for loop by the number of attributes per vertex to get that loop. In practice, I had expanded the range by one and a half length of attributes and noticed that the full stride was still displayed. It should have been half a stride of out of bound data displayed.

Furthermore, the first 2 pieces of data were consistently 0. I thought this meant there was 2 extra elements in the array and spent a good deal of time

looking at the class definitions but found nothing abnormal. It could just be another wierd memory moment.

My main concern is that in the vertex buffer object data displaying function. The loop seemingly reads an entire stride before stopping. No stopping in between. Other wise the previous test would have stopped at half the elements out of bound displayed. Also tried one element out of bound and the loop still displayed every element for that stride. I don't know how this could be possible considering the logic and operations seem simple enough. I have gone over them multiple times and found nothing unusual.

Tommmorow I will investiga this phenomenon.

2026/5/24

15:05:40

I got the display to output the 3d form now. One of the parameters of the vertex array object was entered as the spheres vertices per ring instead of total vertices which was an easy fix. I also fixed the vertex data dump not displaying the final vertex. I think it has something to do with the terminal? I just inserted an endlime at the end of the function and now the function displays the data correctly. This did take a while to figure out though.

The main issue right now is generating the "prime vertices". The vertices that all vertices in the first and last vertex ring would connect to. I tried a few different things before I landed on what I believe now is the right method. I will first generate all the vertex rings. Then I will append the vertex data of the prime vertices to the generated donut. There is no way to actually append to the classic type of array I am using. If I had used a vector type I could append, but I was not too familiar with the "behind the scenes of how vector type arrays expand and shrink. I would rather have full control of memory.

To append I will create a new array with space for two extra vertices data. I will skip over the first vertex data spot and copy over the whole generated array. This leaves an empty stride of elements at the start and end of the array. Then I will simply add the data. The data can be hardcoded since The prime vertices position won't change according to my understanding.

The tough part right now is trying to make the vertex rings generate for both lone and paired prime vertex structures. I will make seperate functions for both types if I can't figure it out by the end the day.

17:28:46

The vertex sphere is successfully generated. Any input number that is a power of 2 should work flawlessly. It's very pretty. It should be noted since I had not implemnted any multitasking in this process, increasing the input number will exponentially increase processing time.

I was going to implement a second algorithm to fit the behaviour even non powers of 2 even numbers. However now I feel as there is no need. Even though the algorithm would be very simple, it would produce a non-spherical object with the current algorithm. There would be a face on both prime vertices.

I tried the program with 1024 as the input and got a almost perfect sphere. My starting concerns yielded true, it was impossible to discern the objects depth. This would make the program look like perfect circles colliding in 3D space. I believe it might look a bit uncanny/unreadable, thus I will seriously consider adding lighting to the project. I hoped it wouldn't come to this mainly because I am not confident my computer can handle it.

With the rendering parts end in sight (atleast a little), I've had more time to think about the physics side of things and concluded another challenge. The collision detection. Although the algorithm is simple. I can see that with more particles, the number of calculations that must be done per frame will be grow very large. Since currently, the algorithm involves every particle comparing its world space origin to every other particles world space origin. This will lead to some duplicated calculations, reducing the efficiency of the algorithm. I am also not too sure where these calculations done. If the compiler is helping out by throwing these calculations in the GPU or I can figure out a way to do it with the GPU, then it would be fine. The collision calculations and lightning calculations may very quickly drown the CPU.

I currently haven't thought up many solutions since this issue is still a bit far along the road. My current ideas involve calculating per a selected number of frames, multithreading, grouping particles based on quadrants and velocity direction.

Next I have to generate the EBO.

2026/5/25

17:50:31

Today I fixed the radius property of the sphere, finished implementing the program initialization procedure with all its validations and started work on the ebo (element buffer object).

I caught the radius error when implementing the manual entry procedure. Any radius value that was greater than 1 generated a cuboid. At first I thought it was because I was scaling the vectors wrong but after some math testing it turned out fine. The real culprit was the floating point precision error corrector. It was setting values greater than 0.9999 to 1. I only tested numbers less than 1 so I did not come across this bug before. The fix was simply scaling the functions boundaries and correction values to the radius.

I was able to figure out all the mathematical parts of the ebo generator algorithm relatively quick compared to figuring out the vbo algorithm. Probably because the numbers were nicely arranged in a anticlockwise spiral.

I had implemented the vbo properties in a class before for the sake of readability but aside from that, writing code is so much easier with the class autofilling everytime and only having to pass one thing in. I am assuming there is a performance trade off but it is one I am willing to pay for the convenience. I also implemented the ebo and its properties in this class so I should probably name it to something broader. I also heard it was better practice to declare classes in separate c files and include them with header files. The class is currently not that large (about 25 lines) so I won't implement it yet. Maybe if the class grows larger or when I standardize the projects file structure.

Next I have to finish the ebo generation algorithm.

2026/5/27

11:43:59

Yesterday I finished the algorithm to generate ebo data for both prime vertices.

Today I spent the whole morning trying to make the algorithm fill in between the layers. Currently the algorithm fills in every square by square which forced me to use a lot of hard coded numbers. This made the algorithm harder to read and diagnose.

Next session I will change the algorithm to first generate all the first triangles in each rectangle. Then pass another wave generating the second triangle, filling in the squares.

This method's only downside is that the ebo data would be more fragmented. However this method will allow me to more seamlessly implement multithreading into the algorithm if I ever decide to. I believe it would look cleaner too.

Before I start implementing the algorithm, I need to first define some variables since today's attempts have all had very long lines mostly due to all the object accessing in expression.

2026/5/28

15:14:55

I finished the element buffer data generator. Now the sphere can be fully rendered using the element buffer object to draw. It works as expected under all validated parameters.

Now I was finally able to practically see how well the depth of the particles could be perceived. As expected it was impossible to discern depth. Due to this, when lesser vertices per ring counts were used (64), the outline seemingly morphed when rotating, causing a buggy visual effect.

I currently have thought of only 2 solutions. The first simpler solution is to change the colour of the vertices. Perhaps a randomized shade of a colour per vertex. This would allow the depth to be perceived but the particles would look "dirty". A cleaner approach is to implement basic lighting. I will have to learn

how to implement lighting to do that. I approximate it would take me 2 weeks, since AS classes are about a week away from starting.

While I learn how lighting, I do want to optimize the code. I will do this by making the data generators more efficient. To do this I may need to learn some X86 assembly to reduce machine code instructions/use less demanding commands. I will also try to implement multithreading.

I am not too worried that I won't be able to learn lighting since "learningopengl.com" is such a great resource. However, I feel like I might be taking on too many things at once. Lighting, Optimizing, ESP-IDF, C programming and recently I have gained an interest in OS development. I am currently writing another program to keep track of all the projects I am currently working on. I will implement a deadline feature so I can tell if I am working too slow compared to what I expected. This will help me decide if I have to cut down on certain work to prioritize this.

2026/6/2

17:50:40

I finished learning the necessary components from "learnopengl.com" to implement a simple lighting system in about 2-3 days. I learnt up to basic lighting where I learnt how to implement phong lighting. I will be using diffuse lighting since it should do the job of providing contrast for depths and the phong implementation was not very smooth for me. The spectral effect was rendering on every face of the object and not just the face facing the light source.

Currently I am mostly done implementing the system in this project. The shader programs are written and the vertex normal attribute of the vertices are calculated and stored as the second vertex attribute instead of color. The color of objects are now defined by uniform variables in the fragment shader.

I thought calculating the vertex normal of the spheres vertices would be a huge challenge but then I realized it was basically just the direction a vector from the origin to the vertex would point in. I was already using a vector to calculate the vertices position so I could just normalize it to get normal data.

I also did a little optimization. I implemented the calculation of the layers radius once per layer and wrote a better Cmake command to link the shader programs. Before cmake copied the shader file to the executable. This sometimes caused the shader program used while running the program to be of a older version than the current. Now Cmake defines an environment path variable that the shader objects construction uses to locate the files. I also planned some more code optimization like multithreading and using assignment operations as much as possible instead of arithmetic.

I have also been learning Cmake instead of esp-idf (I am waiting on a IC to arrive) so I should be able to clean up the project directory to be more standard soon.

Next I have to initialize buffer objects for the light source and assign changing uniform variables in the shaders in the render loop to get the actual final rendered sphere.

2026/06/14

0.1 14:57:39

I am done implementing the diffuse lighting system. it works as expected with all tested vertex counts and under all tested translations. The rendering part of the program is fully done (unless a bug pops up which ofcourse will happen).

I first have to implement a system to easily create particle objects.

2026/06/04

12:57:44

I am currently planning on how the particle attribute heirarchy will go to differentiate between particles, particle types and buffer object types. Lots of design choices to make.

The biggest design choice is if I should allow more than one set of buffer objects to exist. This will grant another attribute that can be used to differentiate particle types. However now that I think about it. The difference between particles vertices count would be very hard to notice. Since smoothness of the sphere exponentially changes. So this would only work if there were particles with very low vertices count. This would look bad though.

I think allowing only one buffer object at runtime would suffice. Differentiating between particles can be done only using different colors and particle size. This should still allow for more than enough different particles to be expressed. This would also allow for a easier implementation of runtime vertex count modification to change the performance.

2026/06/05

13:25:33

I finished implementing the usage of multiple particles. Each particle is a instance of the class currently called class particle. The objects are all elements in a pointer array of the same type. The particles are linked to a particle type class, Allowing multiple types of particles to exist if desired. The particle type class is linked to the buffer objects class.

I can finally start implementing physics. I will first start off by making all the hitboxes work. In the program there should only be particles collision with each other and particles collision with the wall. I will implement the latter first. At the same time I will try to make the project directoy more tidy.

14:42:12

I have thought about how to make the cube boundry and settled on taking the cube's area as input and converting it into a system of 12 linear inequalities. I am sure I can significantly optimize this section by reducing the number of equations to I believe just one or 2 due to the nature of cubes.

2026/06/06

19:11:30

I spent the whole day refactoring the code. I successfully moved the glad file to another directory called external. I also changed the CMakeLists file all by myself, I am finally starting to get it.

After moving the glad, next I wanted to split the classes into their own header and implementation files. However, when I implemented it a very difficult issue occurred. The element buffer object would get populated mostly normally but 1 to 3 elements belonging the first prime vertex would be assigned a very high number. The specific element and assigned number changes across application runs so it is a runtime issue. The element also seems to always be the first element of the triangle. I double checked the class in question (the buffer objects class) and its split implementation and header file. There were no issues I could find. When I used the class in the main file this issue did not happen.

The strangest part is that when I include the implementation file in the cmake add executable command but don't "include" the header file on the main file, the issue still occurs when the main program should be using the fine original class in the main file. This is only fixed when I exclude the implementation from the cmake command.

I tried using deepseek a bit but it very quickly started hallucinating, giving clearly wrong diagnosis. Along with this issue I am sure I will come across a few other issues while refactoring. Furthermore, my AS is starting tomorrow so I can't just spend the whole day developing this. Thus, I am giving myself by the most one week to refactor and standardize this project before I start implementing new features.

2026/06/08

14:50:36

In the past 2 days I finished up separated the classes into separate header and implementation files. I am still not sure what the issue was with my first problem. I solved it yesterday by reverting back to my last commit and rewriting everything.

After that I thought I had an issue with include files not being found. I originally thought it was saying the main file could not find the includes but

then later when I checked with DeepSeek, It pointed out that the particle class files were the ones unable to find the buffer objects class. If I had read the error a bit more carefully I probably could have solved it much faster. The solution was to simply include the libs directory to the particle library target.

Next I will refactor the functions to be seperate files, grouped based on their common function.